MAT61-2

Mathematical Modeling & Decision Science

Formulation, Uncertainty, Sensitivity, and Validation

Translate ambiguous problems into explicit mathematical structures, quantify the uncertainty, and test robustness before you recommend action.

Albert School — 30 hours

Contents

Preface

You arrive at this capstone already fluent in the pieces. From **Probability & Statistics II** (MAT31-2) you can compute with continuous random variables, you know the normal distribution and the Central Limit Theorem, you can build confidence intervals and test hypotheses, and you have met the Markov property and the Bellman equation on a single small example. From **Applied Probability & Simulation** (MAT31-5) you can turn a target quantity into an expectation $E[g(X)]$, estimate it by Monte Carlo, attach a standard error, reduce variance, bootstrap a confidence interval, and read a two-parameter likelihood surface. Running alongside you this semester, **Math for ML** (MAT61-3) sharpens the same instinct for the bias–variance trade-off and out-of-sample error that we will reuse for validation, and the paired **Stochastic Calculus** course (MAT61-4) gives us geometric Brownian motion as a continuous-time uncertainty model.

This course does not add a new computational technique so much as it imposes a **discipline**. A decision maker hands you a paragraph of English — vague, incomplete, contradictory — and expects a defensible recommendation. The work is to turn that paragraph into an explicit mathematical object, to be honest about what you do not know, to find out which of your guesses actually matter, and to state plainly the conditions under which your advice should be ignored. Every chapter is one stage of that pipeline:

formulate (Ch. 1–3) → **quantify uncertainty** (Ch. 4–7) → **test robustness** (Ch. 8) → **validate** (Ch. 9) → **recommend** — with strategic interaction (Ch. 10) and the capstone dossier (Ch. 11) tying it together.

Throughout, we favour the concrete: each idea appears first as a small, fully worked situation and only then as a definition. The mathematics is ordinary; the rigour is in the honesty.

1 Model formulation

> **Worked example — A bakery's morning decision.** A bakery can bake croissants and pains au chocolat. Each croissant uses 50 g of butter and 0.1 h of oven time and earns € 1.20 of margin; each pain au chocolat uses 80 g of butter and 0.15 h and earns € 1.80. This morning there are 6 kg of butter and 10 h of oven time. How many of each should they bake?

> Notice what we did before any equation: we decided that the **things we control** are the two quantities baked; that **success** means total margin; that the butter and oven are **limits**; and we silently **assumed** every croissant sells. Those four moves — variables, objective, constraints, assumptions — are the whole of formulation.

A decision model is the translation of a verbal problem into an explicit mathematical structure. Formulation is the most important and most error-prone stage of the entire pipeline, because every later stage inherits its mistakes.

Definition 1 (Decision model) A decision model consists of:

- **Decision variables** $x = (x_1, ..., x_n)$ — the quantities the decision maker controls.
- **Objective function** $f(x)$ — a scalar measure of outcome to be maximized or minimized.
- **Constraints** $g_i(x) \leq b_i$ — the conditions a feasible decision must satisfy.
- **Assumptions** — the documented simplifications under which the model is taken to represent reality.

For the bakery, with x_1 croissants and x_2 pains au chocolat:

$$\max_{x_1, x_2} \quad 1.20x_1 + 1.80x_2 \quad \text{s.t.} \quad 0.05x_1 + 0.08x_2 \leq 6, \quad 0.10x_1 + 0.15x_2 \leq 10, \quad x_1, x_2 \geq 0.$$

Three model **families** recur in this course, distinguished by what kind of structure the problem has:

- **Linear programs (LP)** — a linear objective optimized over linear constraints; for static resource-allocation decisions (Chapter 3).
- **Ordinary differential equations (ODE)** — a state evolving in continuous time, $y' = f(t, y)$; for dynamic growth/decay/saturation decisions (Chapter 2).
- **Stochastic decision models** — outcomes depend on random quantities whose distributions must be specified (Chapters 4–7).

A disciplined formulation labels every symbol as exactly one of three kinds: a **decision** (you choose it), a **parameter** (a fixed input, to be stress-tested later in Chapter 8), or a **random variable** (you specify its distribution, in Chapters 4–7). Confusing these three is the most common formulation error.

Common pitfall The dangerous assumptions are the ones you do not write down. "Every croissant sells", "butter is the only scarce input", "margins do not change with volume" are all **modelling choices**, not facts. Each one you fail to record is an assumption the validation memo (Chapter 9) cannot later test, and therefore a way your recommendation can fail silently. Write the assumption list **first**, while you still notice the simplifications.

Remark Formulation is not unique. The same bakery could be modelled with integer variables (you cannot bake half a croissant), with uncertain demand, or dynamically over a week. Choosing the **simplest** model that still answers the decision is itself a skill — and Chapter 9 will tell you whether you chose well.

2 Dynamic modeling with ODEs

Some decisions hinge not on a static allocation but on **how a quantity evolves**: a user base that grows, a tax debt that compounds, a reservoir that fills toward capacity. When the rate of change depends on the current state, the natural language is the ordinary differential equation.

> **Worked example — When does the market saturate?.** A new app has 5 thousand users and is growing. Early on it grows almost proportionally to its size (word of mouth), but the addressable market caps at $M = 100$ thousand, and growth slows as it fills. A clean model of "growth proportional to current size **and** to remaining room" is
>
> $$y'(t) = 0.5\, y(t)\left(1 - \frac{y(t)}{100}\right), \quad y(0) = 5.$$
>
> Marketing wants to know roughly **when** the app passes 80 thousand users, to time a funding round. We do not need a formula for $y(t)$ — we need the **trajectory**, and the crossing time.

Definition 2 (Initial value problem) A continuous process whose rate of change depends on the current state is modeled as an **initial value problem**

$$y'(t) = f(t, y(t)), \quad y(t_0) = y_0.$$

The three canonical shapes are

$$y' = ky \quad \text{(growth/decay)}, \quad y' = ky\left(1 - \frac{y}{M}\right) \quad \text{(logistic saturation)}, \quad y' = k(M - y) \quad \text{(bounded approach)}.$$

The function f encodes the mechanism; y_0 pins down the starting point. We deliberately ask for **no analytical solution**. The professional skill is to simulate the trajectory numerically and read the decision off it.

2.1 The Euler method

> **Worked example — One step by hand.** Take $y' = 0.5y\left(1 - \frac{y}{100}\right)$ at $y_0 = 5$. The instantaneous slope is $f(0, 5) = 0.5 \cdot 5 \cdot (1 - 0.05) = 2.375$ thousand users per unit time. If that slope held for one full time unit, we would land at $5 + 1 \cdot 2.375 = 7.375$. It does not hold exactly — the slope changes as y moves — but over a **short** step h it is nearly constant, and the error shrinks with h. That is the entire idea of Euler's method.

Definition 3 (Explicit Euler method) Given step size h and $t_{n+1} = t_n + h$, the explicit Euler method follows the tangent line for one step:

$$y_{n+1} = y_n + h\, f(t_n, y_n).$$

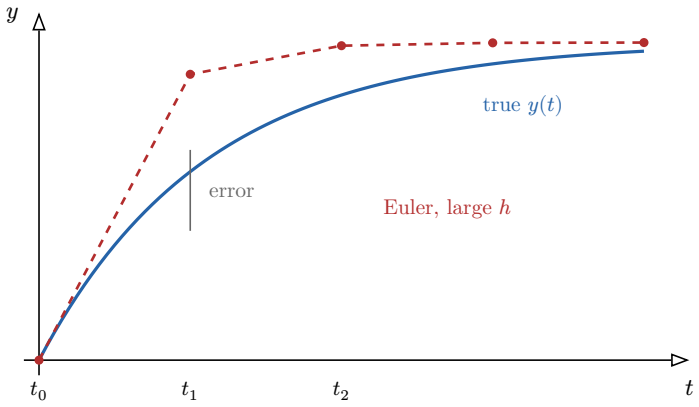

Figure 1: Euler's method follows the tangent at each point. With a large step h, the straight segments drift away from the curving true solution; the gap is the accumulated truncation error. Halving h roughly halves it.

In Python the method is a short loop:

```
import numpy as np

def euler(f, t0, y0, h, n):
    t, y = t0, y0
    ts, ys = [t0], [y0]
    for _ in range(n):
        y = y + h * f(t, y)        # one tangent step
        t = t + h
        ts.append(t); ys.append(y)
    return np.array(ts), np.array(ys)

# Saturating growth y' = 0.5 y (1 - y/100), y(0) = 5
f = lambda t, y: 0.5 * y * (1 - y / 100)
ts, ys = euler(f, 0.0, 5.0, 0.05, 400)     # up to t = 20
cross = ts[np.argmax(ys >= 80)]            # first time y reaches 80
```

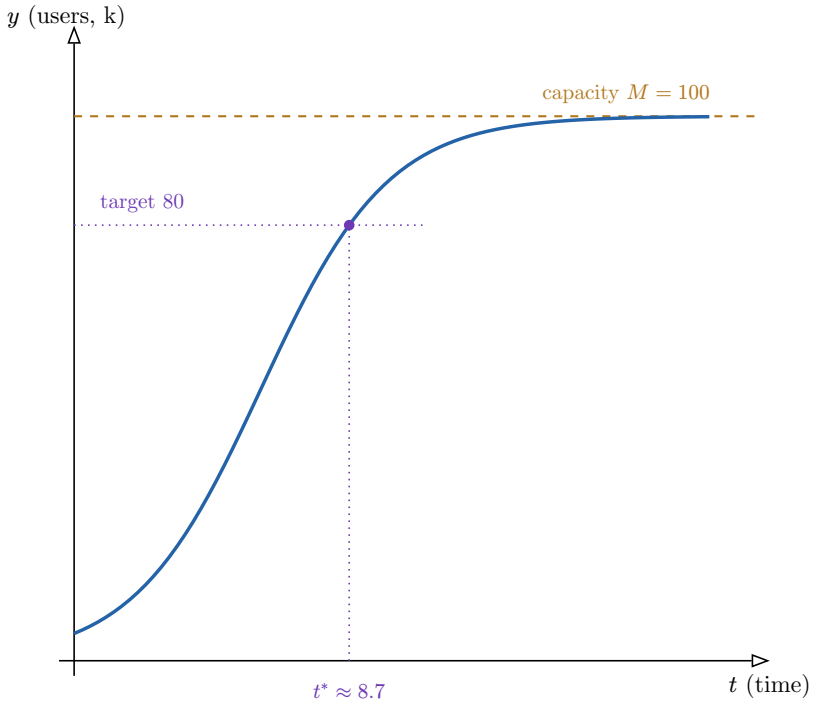

Figure 2: The simulated logistic trajectory. The decision-relevant quantity is the crossing time t^* at which the user base first reaches 80 thousand — read directly off the numerical solution, no closed form needed.

Common pitfall Explicit Euler accumulates error of order h per unit time and can go **unstable** — oscillating or blowing up — when h is too large relative to the rate k (roughly, when hk approaches 2). Always rerun with h halved: if the trajectory, and the decision you read off it, do not change materially, the step is small enough. If they do change, h was too coarse and your earlier answer was an artefact of the discretization, not the model.

3 LP-based decision models

We return to the static allocation problems of Chapter 1 and solve them properly. Linear programming is the workhorse of resource allocation, transportation, production planning, and staffing — and, crucially for a **decision** course, it hands you the marginal value of every scarce resource for free.

Worked example — The bakery, solved and interpreted. Recall the bakery's program (rescaling butter to kilograms):

$$\max \quad 1.2x_1 + 1.8x_2 \quad \text{s.t.} \quad 0.05x_1 + 0.08x_2 \le 6 \,(\text{butter}), \quad 0.10x_1 + 0.15x_2 \le 10 \,(\text{oven}), \quad x_1, x_2 \ge 0.$$

The optimum turns out to use **all** the butter but leaves oven time to spare. The natural follow-up question is not "how much do we earn?" but "what is one extra kilogram of butter worth to us?" That number — the shadow price — is what makes LP a decision tool rather than a calculator.

Definition 4 (Standard form) A linear program in standard form is

$$\max_{x} \quad c^\top x \quad \text{subject to} \quad Ax \le b, \quad x \ge 0,$$

where x are activity levels, c are per-unit returns, and each row of $Ax \le b$ is a resource limit with right-hand side b_i.

The feasible set is a convex polygon (a polytope in higher dimensions), and the optimum, if it exists, is attained at a **vertex**. Geometrically, you push the objective's contour line as far as it will go in the improving direction; it last touches the feasible region at a corner.

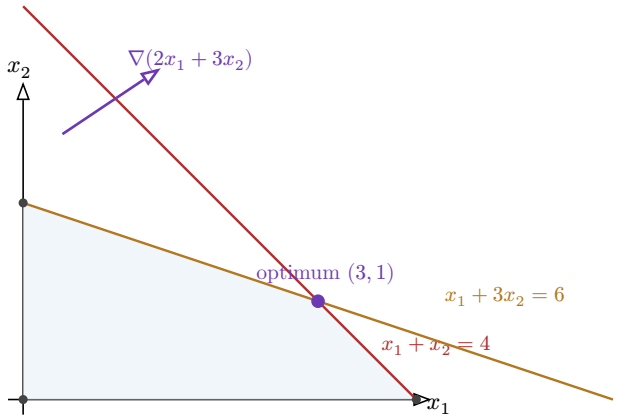

Figure 3: A two-variable LP. The shaded polygon is the feasible region; the objective gradient (purple) points in the improving direction. The optimum sits at the vertex where both constraints bind — those are the resources with positive shadow prices.

Definition 5 (Dual variable and shadow price) Each constraint i carries a **dual variable** $y_i \ge 0$. At the optimum, y_i is the **shadow price** of resource i: the rate of change of the optimal objective value per unit increase of the right-hand side b_i,

$$y_i = \frac{\partial \text{ (optimal value)}}{\partial b_i},$$

valid over the range of b_i for which the optimal vertex (the basis) does not change.

The shadow price is the bridge from mathematics to management. A **binding** constraint ($A_i x = b_i$ at the optimum — the resource is fully used) generally has a positive shadow price: loosening it improves the objective. A **non-binding** constraint, with slack left over, has shadow price zero — more of that resource is worthless right now. So the shadow price is exactly the marginal value of the resource and is the ceiling on what you should pay to acquire one more unit.

Worked example — Reading the bakery's duals. Suppose the solver reports a butter shadow price of € 22 per kg and an oven shadow price of € 0. Interpretation: oven time is **not** the bottleneck (slack remains), so buying an extra oven hour earns nothing; but a kilogram of butter is worth up to € 22 of additional margin — pay anything below that to source more, and stop trusting the € 22 once you have bought enough butter to make a different constraint bind.

Common pitfall A shadow price is **local**. It holds only within the range over which the same set of constraints stays binding. Multiplying it by a large quantity ("butter is worth € 22, so 1000 kg more is worth € 22{,}000") is wrong: somewhere along the way oven time, or demand, becomes the new bottleneck and the marginal value collapses. The valid range comes with the dual; always report it.

Every LP carries an **assumption audit**: linearity of returns and costs, divisibility of x (fractional activities allowed), certainty of the coefficients in A and c, and additivity (no interaction between activities). Each assumption that fails — integer-only batches, volume discounts that bend the objective, uncertain yields — flags a limit on the model's validity that Chapter 9 must address.

4 Decision under uncertainty

LP assumed every number was known. Most real decisions are made against an unknown **state of nature**. We index actions by a and states by s and collect the outcomes in a payoff matrix $V(a, s)$. With no sequential structure yet, the question is purely: which row do you pick?

> **Worked example — The food-truck pitch.** A vendor must choose how much stock to prepare for a festival: **Low**, **Medium**, or **High**. Profit depends on turnout, which is **Poor**, **Fair**, or **Great**:
>
> | **action \ state** | Poor | Fair | Great |
> |---|---|---|---|
> | Low stock | 200 | 200 | 200 |
> | Medium stock | 80 | 320 | 320 |
> | High stock | −120 | 240 | 560 |
>
> Three reasonable decision makers will pick three different rows — and each can defend the choice. The criteria below make those instincts precise.

Definition 6 (Expected value (EV)) If state s occurs with probability $p(s)$, the expected value of action a is

$$\mathrm{EV}(a) = \sum_{s} p(s)\, V(a, s),$$

and the EV criterion selects $a^* = \arg\max_a \mathrm{EV}(a)$.

Definition 7 (Maximin / minimax) Without trustworthy probabilities, the **maximin** criterion picks the action whose worst outcome is best:

$$a^* = \arg\max_{a} \min_{s} V(a, s).$$

For a **loss** matrix the symmetric criterion is **minimax**: minimize the worst-case loss.

Definition 8 (Minimax regret) The **regret** of action a in state s is what you forgo versus the best action for that state,

$$R(a, s) = \max_{a'} V(a', s) - V(a, s),$$

and the minimax-regret criterion selects $a^* = \arg\min_a \max_s R(a, s)$.

> **Worked example — Same matrix, three answers.** Take probabilities $p = (0.25, 0.50, 0.25)$ over (Poor, Fair, Great).

- **EV**: Low $= 200$; Medium $= 0.25(80) + 0.5(320) + 0.25(320) = 260$; High $= 0.25(-120) + 0.5(240) + 0.25(560) = 230$. EV chooses **Medium**.
- **Maximin**: worst outcomes are $200, 80, -120$. Maximin chooses **Low**.
- **Minimax regret**: the regret table's worst-case regrets are 360 (Low), 240 (Medium), 320 (High). Minimax regret chooses **Medium**.

The cautious actor stocks Low; the actor who trusts the odds, or who fears hindsight, stocks Medium.

When is each appropriate? **Expected value** when the probabilities are credible **and** the decision repeats, so averages are meaningful. **Maximin** under deep uncertainty or strong risk aversion, where only the worst case matters. **Minimax regret** when you will be judged after the fact against the best you could have done.

Common pitfall Do not average over states whose probabilities you invented. Expected value is only as trustworthy as $p(s)$; applying it to "probabilities" pulled from nowhere launders a guess into a number. If the $p(s)$ are unknown, that is itself the finding — use maximin or regret, and report that you avoided EV **deliberately**.

Definition 9 (Expected value of perfect information (EVPI))

$$\text{EVPI} = \underbrace{\sum_s p(s) \max_a V(a,s)}_{\text{EV with a perfect forecast}} - \underbrace{\max_a \sum_s p(s) V(a,s)}_{\text{EV of the best single action}}.$$

It is the most you should pay to learn the true state **before** deciding.

Worked example — Is a forecast worth buying?. For the food truck, a perfect turnout forecast lets you match stock to state: $0.25(200) + 0.5(320) + 0.25(560) = 350$. The best action without it earns 260 (Medium). So EVPI $= 350 - 260 = 90$: a market study costing less than € 90 (in these units) could pay for itself; one costing more cannot, **even if perfectly accurate**. Real forecasts are imperfect, so 90 is an upper bound on any study's value.

5 Probabilistic decision trees

EVPI hinted at **sequence**: learn, then act. Decision trees make sequence explicit. They interleave **decision nodes** (squares — you choose) and **chance nodes** (circles — nature chooses, with probabilities), with payoffs at the leaves, and they are solved by working backwards.

Worked example — Launch, test, or drop. A firm can **Launch** a product now or **Drop** it. Launching costs nothing extra to model here: the market is Good (prob 0.6, payoff 100) or Bad (prob 0.4, payoff -20). Dropping yields a safe 30. Should they launch?

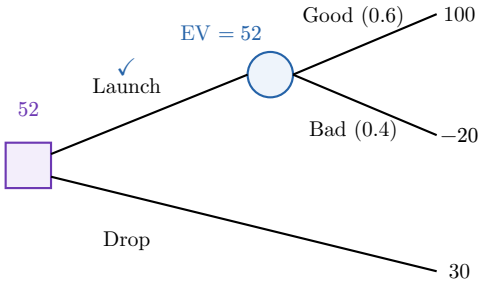

Figure 4: A one-stage decision tree. Fold-back: average at the chance node ($0.6 \cdot 100 + 0.4 \cdot (-20) = 52$), then take the max at the decision node ($\max(52, 30) = 52$). The recommendation is **Launch**, worth 52.

Definition 10 (Fold-back (backward induction)) The optimal policy is computed from the leaves back to the root:

- at a **chance** node, take the **expected value** over its branches;
- at a **decision** node, take the **maximum** over its branches and record the maximizing choice.

The value at the root is the optimal expected payoff; the recorded choices form the optimal **policy**.

This is exactly the Bellman one-step accounting identity you met in MAT31-2 — "value today = best choice today + discounted value of tomorrow" — applied to a finite tree instead of a recurring state.

5.1 Sensitivity of the recommendation

A decision tree gives one answer, but that answer rests on guessed probabilities and payoffs. The professional question is: **how wrong can the guesses be before the recommendation flips?**

Definition 11 (Breakeven value) Vary one probability or payoff, holding the others fixed, and recompute the fold-back. The **breakeven value** of that parameter is the value at which the recommended action changes.

Worked example — How bad can the odds get?. Let p be the probability of a Good market. Launch is preferred while $100p - 20(1-p) \geq 30$, i.e. $120p \geq 50$, i.e. $p \geq 0.417$. So the breakeven is $p^* = 0.417$: as long as you believe the market is good with probability at least about 42%, launch. Our estimate was 0.60 — comfortably clear of the breakeven, so the recommendation is **robust** to moderate misjudgement of p.

A recommendation that flips inside the plausible range of a probability or payoff is **fragile** and must be reported as such; one whose breakeven is far outside the plausible range is **robust**. This distinction is the entire content of Chapter 8, generalized.

6 Simulation-based policy evaluation

Decision trees need a manageable number of branches. When a policy's outcome is a tangle of many random inputs with no closed form — a supply chain with random demand, lead times, and prices — you reach for Monte Carlo, exactly the tool from MAT31-5, now in service of a decision.

Worked example — Will the inventory policy stock out?. A shop reorders 50 units whenever stock drops below 20. Daily demand is random (say Poisson), lead times vary, and you want the probability of a stockout over a quarter, with a confidence interval. There is no formula — so you **simulate** a quarter thousands of times and count.

Definition 12 (Monte Carlo estimate) To evaluate a policy whose outcome is $g(X)$ with X random, draw independent replications $X_1, \ldots, X_N$ and report

$$\hat{\mu} = \frac{1}{N}\sum_{i=1}^{N} g(X_i), \quad \text{SE} = \frac{s}{\sqrt{N}},$$

where s is the sample standard deviation of the $g(X_i)$. An approximate 95% confidence interval is $\hat{\mu} \pm 1.96\,\text{SE}$ (Central Limit Theorem, MAT31-2).

```
import numpy as np
rng = np.random.default_rng(0)

def evaluate(simulate_outcome, N):
    g  = np.array([simulate_outcome(rng) for _ in range(N)])
    mu = g.mean()
    se = g.std(ddof=1) / np.sqrt(N)
    return mu, (mu - 1.96 * se, mu + 1.96 * se)
```

A scenario analysis (i) specifies the scenarios and their generating mechanism, (ii) simulates outcomes under the policy, and (iii) reports the **distribution** of results — not just the mean, but the spread and the confidence interval for the mean. The interval width shrinks at rate $1/\sqrt{N}$, so cutting it in half costs four times the simulation.

Remark Everything from MAT31-5 transfers directly: variance reduction (antithetic or control variates) buys you a narrower interval for the same N; bootstrapping gives intervals when the CLT is shaky; and you must **validate** the simulator against an analytical special case before trusting it (Chapter 9).

Common pitfall Reporting $\hat{\mu}$ without its confidence interval is the cardinal Monte Carlo sin. Two policies whose estimated means differ by less than the interval width are statistically indistinguishable at this N; recommending one over the other on the point estimate alone is reading noise as signal. Always carry the interval into the decision.

7 Continuous-time decision modeling

Chapters 4–6 modelled uncertainty with a **handful of discrete states**. But some decisions live in continuous time against continuously evolving uncertainty — a price, an exchange rate, a demand level that drifts and fluctuates every instant — and, crucially, they are **irreversible**: once you build the plant you cannot unbuild it. Here the **option to wait** has value.

Worked example — Invest now or wait a year?. A firm can build a plant for a fixed cost; its payoff depends on a commodity price S_t that wanders unpredictably. Building now locks in today's uncertain prospects. Waiting a year sacrifices a year of cash flow but reveals where the price went — and lets you **not** build if it collapsed. That right to walk away is

worth something, and a naive expected-value calculation that ignores it will systematically over-invest.

Definition 13 (Geometric Brownian motion (GBM)) The standard continuous-time uncertainty model, from the paired Stochastic Calculus course (MAT61-4), is geometric Brownian motion

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t,$$

with drift μ, volatility σ, and W_t a Brownian motion. Its scenarios S_t are simulated and fed into the decision; S_t stays positive and its log grows linearly with normally distributed fluctuations.

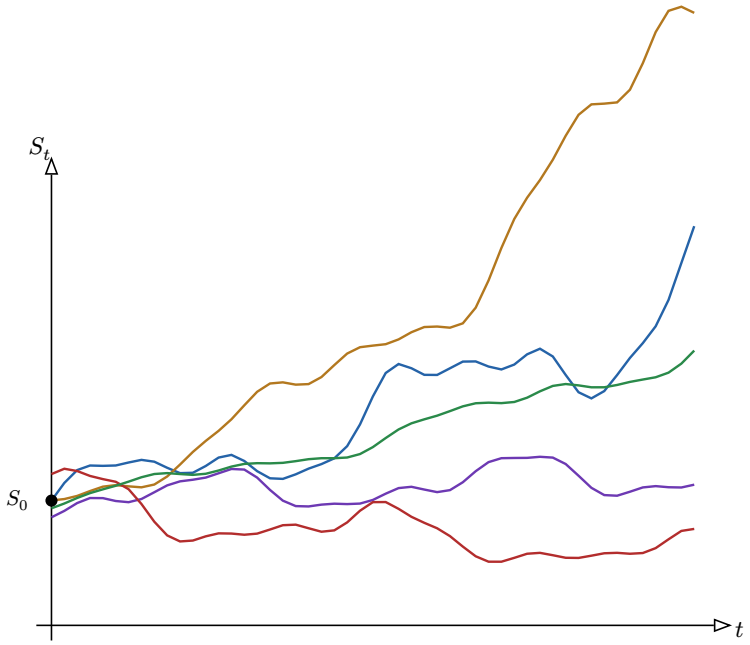

Figure 5: Five simulated GBM scenarios from a common start S_0. The ensemble fans out over time: continuous-time models capture the full, widening range of futures — and hence the value of waiting to see which path materializes — that a few discrete states cannot.

Real-option framing. An irreversible decision carries an **option value**: the right, but not the obligation, to act later once more uncertainty has resolved. Acting immediately forgoes that option. The comparison this course asks you to make is between two uncertainty models on the **same** decision:

	Discrete-scenario	**Continuous-time (GBM)**
structure	few states, assigned probabilities	ensemble of simulated paths
evaluated by	EV / decision tree (Ch. 4–5)	Monte Carlo simulation (Ch. 6)
strengths	transparent, easy to audit	captures timing / option value
weaknesses	coarse, may miss option value	more assumptions: μ, σ, GBM form

The discrete model is transparent and auditable but coarse; the continuous-time model captures the value of waiting at the cost of three extra assumptions (μ, σ, and the GBM form itself). A good dossier runs **both** and reports where they agree and where they diverge.

Common pitfall GBM assumes log-normal returns with constant volatility — no jumps, no fat tails, no regime changes. For decisions exposed to crashes or structural breaks, GBM understates extreme outcomes and the recommendation may be fragile in exactly the scenarios that matter most. State this assumption as loudly as you state the result.

8 Robustness analysis

Chapters 5 and 8 share one obsession: a recommendation is only as good as its inputs. Robustness analysis systematizes the breakeven idea of Chapter 5 across **all** parameters at once, asking how the optimal recommendation responds to perturbations of the things you merely estimated.

Worked example — Which guess sinks the plan?. Your bakery recommendation depends on the butter price, the margins, demand, and the oven capacity. Three of those you measured precisely; one you guessed. Before defending the plan, you want to know: of all these parameters, **which one**, if wrong by a plausible amount, would actually change what you should do? Spending your verification effort on the others is wasted.

Definition 14 (Perturbation map) Let $V^*(\theta)$ be the optimal value as a function of the parameter vector θ. The **perturbation map** collects the partial derivatives

$$\frac{\partial V^*}{\partial \theta_j} \quad \text{at the baseline,}$$

ranking parameters by how strongly the optimum responds to each. (For an LP, $\partial V^*/\partial b_i$ is exactly the shadow price of Chapter 3.)

The full analysis has three layers, each catching what the previous one misses:

- **Perturbation map** — compute $\partial V^*/\partial \theta_j$ (analytically, or numerically by finite differences) for every parameter and rank them. This finds the parameters the **value** is most sensitive to.
- **Stress testing** — shock several parameters **jointly** under named scenarios (recession, supply shock), not one at a time. Interactions can make a combination fragile even when each parameter alone looks safe.
- **Breakeven analysis** — for each critical parameter, find the value at which the recommended **action** flips (Chapter 5), and compare it with the parameter's plausible range.

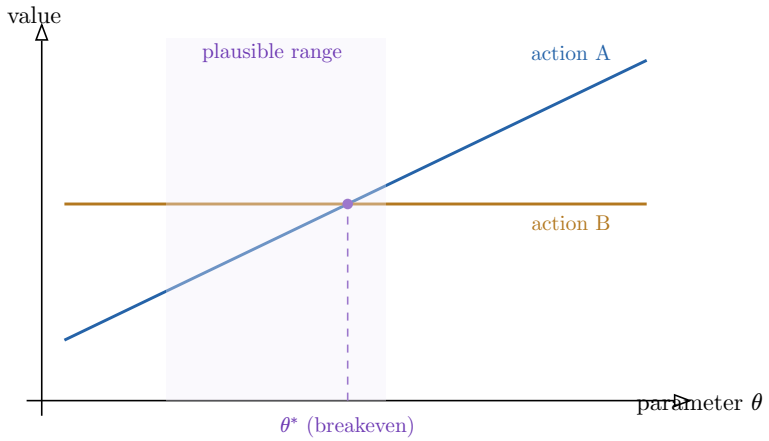

Figure 6: A breakeven plot. Action A overtakes action B at θ^*. Because the plausible range of θ (shaded) straddles θ^*, the recommendation is **fragile** in this parameter: realistic estimation error is enough to flip the decision. If the range lay entirely on one side, the recommendation would be robust.

The deliverable is the **region of parameter space over which the recommendation holds**, plus an explicit list of the **fragile parameters** — those whose plausible variation alone changes the decision. These are precisely the inputs worth spending money to measure better.

Common pitfall One-at-a-time sensitivity can declare a plan safe that joint stress testing reveals as fragile: each parameter alone clears its breakeven, but a recession that moves several adversely **at once** does not. Never stop at the perturbation map; always stress the correlated, worst-case combinations.

9 Model validation

A model that fits the data it was built on proves nothing — it has merely memorized. Validation, in the spirit of the bias–variance and out-of-sample thinking from your concurrent **Math for ML** course, tests the model against data it has **not** seen and diagnoses **model risk**.

Worked example — The backtest that lied. A demand model fit on 2023–2024 predicts last year's sales almost perfectly. Encouraged, you deploy it — and it misses badly. The in-sample fit was an illusion of overfitting; only by holding out a period the model never touched could you have seen it coming. That hold-out test is the heart of validation.

Definition 15 (Back-testing and out-of-sample error) Partition historical data into an **in-sample** period (used to calibrate) and an **out-of-sample** period (held out). **Back-testing** applies the calibrated model over the out-of-sample period and compares predictions $\hat{y}_t$ with realized outcomes y_t. The **out-of-sample prediction error** is a summary such as

$$\text{RMSE} = \sqrt{\frac{1}{m}\sum_{t}(\hat{y}_t - y_t)^2}.$$

Definition 16 (Calibration check) For a **probabilistic** model, group predictions by their stated probability and compare predicted with observed frequency: events assigned

probability p should occur about a fraction p of the time. A systematic gap between the two is **miscalibration**.

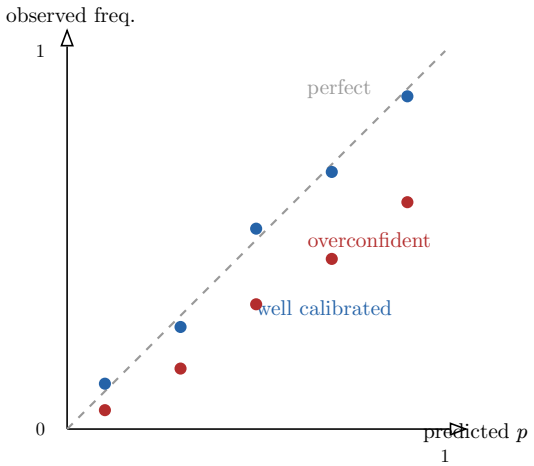

Figure 7: A calibration plot. Blue points hug the diagonal — predicted probabilities match observed frequencies. Red points fall below it at high p: the model is **overconfident**, assigning high probabilities to events that occur less often than claimed. A back-test on accuracy alone would not reveal this.

Diagnosing model risk draws all the threads together:

- compare in-sample and out-of-sample error — a **large gap** signals overfitting (high variance), echoing the bias–variance framework of MAT61-3;
- run the calibration check above for any probabilistic component;
- verify that the **documented assumptions** from Chapter 1 actually held over the validation period.

The output is the **validation memo**: a plain statement of the conditions under which the model — and therefore the recommendation — should **not** be trusted.

Common pitfall Tuning the model until the out-of-sample error looks good turns the hold-out set into a second training set: you have overfit the validation, and the reported error is once again optimistic. Hold out a genuinely untouched period (or use the nested cross-validation discipline of MAT61-3) and look at it **once**.

10 Strategic interaction (minimal core)

Every model so far treated uncertainty as **nature** — indifferent, non-reactive. But sometimes the relevant uncertainty is the deliberate choice of **another rational agent** who is reasoning about you in turn. Then single-agent criteria fail, because the "state of the world" reacts to your action. This chapter is overview scope only: enough to **recognize** when a decision is strategic.

Worked example — Two firms, one price war. Two competitors each choose **Hold** price or **Cut** price. The catch: each one's best move depends on what the **other** does. Modelling your rival as a fixed probability ("they cut with probability 0.5") misses that the rival is choosing on purpose, in response to you. Below, the entries are (your payoff, rival's payoff):

you \ rival	Hold	Cut
Hold	(10, 10)	(2, 12)
Cut	(12, 2)	(5, 5)

Whatever the rival does, you earn more by Cutting ($12 > 10$, and $5 > 2$) — so Cut is **dominant**. By symmetry the rival cuts too, landing both at $(5, 5)$, worse than the $(10, 10)$ they could have shared. No probability model of "nature" would ever have produced this.

Definition 17 (Dominant strategy) A strategy a_i of player i **dominates** $a_{i'}$ if it yields a payoff at least as high against **every** choice of the opponents, and strictly higher against at least one. A **dominant strategy** dominates all alternatives — it is best no matter what others do.

Definition 18 (Nash equilibrium) A strategy profile $(a_1^*, ..., a_n^*)$ is a **Nash equilibrium** if no player can improve their own payoff by unilaterally changing their strategy while the others hold theirs fixed. (In the example above, (Cut, Cut) is the Nash equilibrium.)

A decision model requires game-theoretic reasoning exactly when the binding uncertainty is **another agent's choice** rather than nature's. The strategic structure **constrains** your recommendation: advice that assumes a state of the world the opponents will overturn once they best-respond is not credible. A plan predicated on the rival holding price, when cutting is their dominant strategy, is built on sand.

Common pitfall Do not model a strategic opponent as a fixed probability distribution. Doing so assumes away the very reaction that defines the problem — the opponent re-optimizes against your move, so the "state probabilities" shift the moment you act. Recognizing this need to switch frames is the learning outcome of this chapter.

11 The capstone dossier

The course deliverable assembles the pipeline into one defensible document. It has **four mandatory outputs**, each the product of a stage of the preceding chapters. Read this as the specification of the whole course working together.

1. **Formal model specification** — decision variables, objective, constraints, and explicitly documented assumptions (Chapter 1), in the appropriate family: LP (Chapter 3), ODE (Chapter 2), or stochastic (Chapters 4–7).
2. **Uncertainty model with calibration assumptions** — the distributions or scenarios used (Chapters 4–7, discrete or GBM-continuous), and **how their parameters were calibrated** from data.
3. **Sensitivity / robustness analysis** — perturbation map, joint stress tests, and breakeven values; the **region in which the recommendation holds** and the list of **fragile parameters** (Chapter 8).

4. **Validation memo** — back-testing and out-of-sample results, calibration checks, and an explicit statement of **the conditions under which the recommendation should not be trusted** (Chapter 9).

Remark The fourth output is the one that separates a decision scientist from a calculator. Anyone can produce a number; the discipline of this course is to hand that number over **together with an honest account of when it is wrong**. A recommendation without its conditions of failure is not a recommendation — it is a guess wearing the costume of one.

Exercises

1. (Formulation.) A clinic schedules nurses across three shifts with given per-shift demand, hourly costs, and a rule that no nurse works two consecutive shifts. Identify the decision variables, objective, and constraints, and list three assumptions your formulation makes. Which model family fits, and why?

2. (ODE.) For $y' = 0.3y\left(1 - \frac{y}{50}\right)$, $y(0) = 2$, take one Euler step with $h = 1$ by hand. Then argue, without solving, whether $h = 1$ is small enough, and describe the check you would run.

3. (LP duals.) An LP solver reports that the labour constraint is binding with shadow price 15 and the material constraint has slack. A manager wants to buy 200 extra labour-hours and claims the value is $200 \times 15 = 3000$. State precisely why this may be wrong and what additional information you need.

4. (Criteria.) Using the food-truck payoff matrix (Chapter 4) with $p = (0.4, 0.4, 0.2)$, recompute the EV, maximin, and minimax-regret choices. Then compute the EVPI and state what it means operationally.

5. (Decision tree.) In the launch tree (Chapter 5), find the breakeven Bad- market payoff: holding $p = 0.6$ and the Good payoff at 100, how negative can the Bad payoff become before Drop is preferred?

6. (Monte Carlo.) You estimate a policy's mean profit as 1200 with a 95% interval $[1150, 1250]$ from $N = 10\{,\}000$ runs. A rival policy estimates 1230 with interval $[1180, 1280]$. Can you recommend one over the other? What N would you need to halve each interval?

7. (Robustness.) Explain, with a two-parameter example, how a plan can pass every one-at-a-time breakeven test yet fail a joint stress scenario.

8. (Validation.) A weather model says "70% chance of rain" on 50 days; it actually rained on 20 of them. Is the model calibrated? Sketch the point on a calibration plot and state the direction of the miscalibration.

9. (Strategy.) Modify the price-war matrix (Chapter 10) so that Cut is **no longer** dominant for you. Identify any Nash equilibria in your modified game and explain why a single-agent EV model would mishandle it.

10. (Continuous vs. discrete.) For an irreversible investment, give one concrete reason a GBM-based real-options analysis might recommend **waiting** where a two-state expected-value tree recommends **investing now**. Which assumptions drive the difference?